

AI534 Implementation 1

Team: IA1 8 (Manogna Challoyu, Shanmukha Rangeneni)

Deadline: Sunday, Oct. 13, by 11:59pm

Submission: Submit 1) your completed notebook in ipynb format, and 2) a PDF export of the completed notebook with outputs (the codeblock at the end of the notebook should automatically produce the pdf file).

In this assignment, we will implement and experiment linear regression to predict the price of a house based on features describing the house, using the housing data that you have explored in the warm up assignment.

We will implement two versions, one using the closed-form solution, and one using gradient descent.

You may modify the starter code as you see fit, including changing the signatures of functions and adding/removing helper functions. However, please make sure that your TA can understand what you are doing and why.

First lets import the necessary packages.

```
In [63]: !pip install nbconvert > /dev/null 2>&1
!pip install pdfkit > /dev/null 2>&1
!apt-get install -y wkhtmltopdf > /dev/null 2>&1
import os
import pdfkit
import contextlib
import sys
from google.colab import files
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
# add more imports if necessary
```

Part 0: (5 pts) data and preprocessing

On canvas, we have provided two different data files for this assignment: ia1_train.csv (for training) and ia1_val.csv(for validation). Download them and upload them to your google drive. Then mount the google drive from your google colab notebook:

```
In [69]: from google.colab import drive
drive.mount('/content/gdrive')

train_path = '/content/gdrive/My Drive/AI534/ia1_train.csv' # DO NOT MODIFY THIS. Please make sure your data has
val_path = '/content/gdrive/My Drive/AI534/ia1_val.csv' # DO NOT MODIFY THIS. Please make sure your data has th
```

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).

Now load the training and validation data.

```
In [73]: train_path = '/content/gdrive/My Drive/AI534/IA1_train.csv'
val_path = '/content/gdrive/My Drive/AI534/IA1_dev.csv'

train_df = pd.read_csv(train_path)
val_df = pd.read_csv(val_path)

print("Training Data Sample:")
display(train_df.head())

print("\nValidation Data Sample:")
display(val_df.head())
```

Training Data Sample:

	id	date	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	...	sqft_above	sqft_below
0	7972604355	5/21/2014	3	1.00	1020	7874	1.0	0	0	3	...	1020	0
1	8731951130	6/9/2014	3	2.25	2210	8000	2.0	0	0	4	...	2210	0
2	7885800740	2/18/2015	4	2.50	2350	5835	2.0	0	0	3	...	2350	0
3	4232900940	5/22/2014	3	1.50	1660	4800	2.0	0	0	3	...	1660	0
4	3275850190	9/5/2014	3	2.50	2410	9916	2.0	0	0	4	...	2410	0

5 rows × 21 columns

Validation Data Sample:

	id	date	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	...	sqft_above	sqft_below
0	3211200460	8/6/2014	4	1.00	1520	9800	1.5	0	0	4	...	1520	0
1	4124000320	3/16/2015	3	2.25	1800	15903	1.0	0	0	3	...	1340	0
2	7129302800	12/12/2014	3	1.50	1780	5000	1.0	0	4	4	...	1030	0
3	1392800035	6/18/2014	2	1.00	1240	6400	1.0	0	1	4	...	1060	0
4	2154900040	10/30/2014	3	2.25	2190	8834	1.0	0	0	3	...	1390	0

5 rows × 21 columns

Perform the following preprocessing steps.

1. remove the *ID* column from both training and validation data
2. change *date* into 3 numerical features *day*, *month* and *year*, like in the warm up exercise
3. The feature *yr_renovated* is set to 0 if the house has not been renovated. This creates an inconsistent meaning to the numerical values. Replace it with a new feature called *age_since_renovated*:

```
if yr_renovate != 0
    age_since_renovated = year - yr_renovated
otherwise
    age_since_renovated = year - yr_built
```

1. Normalize all the features using z-score normalization based on the training data. Do not normalize *price* as it is the target. To normalize a feature *x* using z-score normalization, the formula is

$$z = \frac{x - \mu}{\sigma}$$

where μ and σ are the mean and standard deviation of *x* respectively. The normalized feature will have zero mean and unit standard deviation. Note that you should estimate μ and σ for each feature only using the training data and use the same μ and σ to normalize the features for both training and validation data.

```
In [74]: train_df = train_df.drop(columns=['id'])
val_df = val_df.drop(columns=['id'])

for df in [train_df, val_df]:
    df['date'] = pd.to_datetime(df['date'])
    df['day'] = df['date'].dt.day
    df['month'] = df['date'].dt.month
    df['year'] = df['date'].dt.year
    df.drop(columns=['date'], inplace=True)

for df in [train_df, val_df]:
    df['age_since_renovated'] = df.apply(
        lambda row: row['year'] - row['yr_renovated'] if row['yr_renovated'] != 0 else row['year'] - row['yr_built'],
        axis=1
    )
    df.drop(columns=['yr_renovated'], inplace=True)

features_to_normalize = train_df.columns.difference(['price'])

train_mean = train_df[features_to_normalize].mean()
train_std = train_df[features_to_normalize].std()

train_df[features_to_normalize] = (train_df[features_to_normalize] - train_mean) / train_std
val_df[features_to_normalize] = (val_df[features_to_normalize] - train_mean) / train_std
```

Let's do a quick testing of your normalization, please

1. Estimate and print the new mean and standard deviation of the normalized features for the training data --- this should be 0 and 1 respectively.
2. Estimate and print the new mean and standard deviation of the normalized features for the validation data --- these values will not be 0 and 1, but somewhat close

```
In [75]: train_means_normalized = train_df[features_to_normalize].mean()
train_stds_normalized = train_df[features_to_normalize].std()

print("Training Data - Normalized Features")
print("Mean:")
display(train_means_normalized)
print("Standard Deviation")
display(train_stds_normalized)

val_means_normalized = val_df[features_to_normalize].mean()
val_stds_normalized = val_df[features_to_normalize].std()

print("\nValidation Data - Normalized Features")
print("Mean (should be close to 0):")
display(val_means_normalized)
print("Standard Deviation (should be close to 1):")
display(val_stds_normalized)

print("\nTraining Data Sample (after normalization):")
display(train_df.head())

print("\nValidation Data Sample (after normalization):")
display(val_df.head())
```

Training Data - Normalized Features
Mean:

	0
age_since_renovated	-9.681145e-17
bathrooms	-1.634248e-16
bedrooms	1.652012e-16
condition	6.306067e-17
day	-2.142730e-17
floors	4.396483e-17
grade	1.207923e-16
lat	-4.131451e-14
long	9.379608e-14
month	-2.264855e-17
sqft_above	-9.947598e-17
sqft_basement	5.329071e-18
sqft_living	9.059420e-17
sqft_living15	6.039613e-17
sqft_lot	-1.776357e-18
sqft_lot15	-1.776357e-18
view	-6.394885e-17
waterfront	2.464695e-17
year	1.991887e-13
yr_built	3.400835e-15
zipcode	3.704059e-14

dtype: float64
Standard Deviation

	0
age_since_renovated	1.0
bathrooms	1.0
bedrooms	1.0
condition	1.0
day	1.0
floors	1.0
grade	1.0
lat	1.0
long	1.0
month	1.0
sqft_above	1.0
sqft_basement	1.0
sqft_living	1.0
sqft_living15	1.0
sqft_lot	1.0
sqft_lot15	1.0
view	1.0
waterfront	1.0
year	1.0
yr_built	1.0
zipcode	1.0

dtype: float64

Validation Data - Normalized Features
Mean (should be close to 0):

	0
age_since_renovated	-0.023726
bathrooms	-0.011648
bedrooms	0.005582
condition	-0.055403
day	-0.017296
floors	0.022541
grade	0.057390
lat	-0.018752
long	0.016222
month	-0.017980
sqft_above	0.028274
sqft_basement	-0.024773
sqft_living	0.013909
sqft_living15	0.051889
sqft_lot	0.005044
sqft_lot15	-0.005096
view	0.002659
waterfront	0.015265
year	0.009397
yr_built	0.018428
zipcode	-0.030558

dtype: float64

Standard Deviation (should be close to 1):

	0
age_since_renovated	0.999477
bathrooms	1.003430
bedrooms	0.893240
condition	0.976223
day	1.008404
floors	0.992549
grade	1.033680
lat	1.000963
long	0.986680
month	0.983898
sqft_above	0.996623
sqft_basement	0.981484
sqft_living	0.993781
sqft_living15	1.051790
sqft_lot	0.912514
sqft_lot15	0.785337
view	1.025579
waterfront	1.088181
year	1.003819
yr_built	1.000355
zipcode	1.006450

dtype: float64

Training Data Sample (after normalization):

	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	...	z
0	-0.388520	-1.465613	-1.159110	-0.171139	-0.922332	-0.082432	-0.304487	-0.634184	-0.563015	-0.924206	...	0.
1	-0.388520	0.169158	0.145003	-0.168133	0.917732	-0.082432	-0.304487	0.888924	0.290361	0.507090	...	-1.
2	0.649956	0.496112	0.298428	-0.219787	0.917732	-0.082432	-0.304487	-0.634184	0.290361	0.675478	...	-0.
3	-0.388520	-0.811705	-0.457738	-0.244481	0.917732	-0.082432	-0.304487	-0.634184	0.290361	-0.154433	...	0.
4	-0.388520	0.496112	0.364182	-0.122419	0.917732	-0.082432	-0.304487	0.888924	1.997115	0.747644	...	-0.

5 rows × 22 columns

Validation Data Sample (after normalization):

	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	condition	grade	sqft_above	...	z
0	0.649956	-1.465613	-0.611163	-0.125186	-0.002300	-0.082432	-0.304487	0.888924	-0.563015	-0.322821	...	-0.
1	-0.388520	0.169158	-0.304313	0.020425	-0.922332	-0.082432	-0.304487	-0.634184	0.290361	-0.539319	...	-0.
2	-0.388520	-0.811705	-0.326231	-0.239710	-0.922332	-0.082432	5.014070	0.888924	-0.563015	-0.912178	...	0.
3	-1.426996	-1.465613	-0.918013	-0.206307	-0.922332	-0.082432	1.025152	0.888924	-0.563015	-0.876095	...	0.
4	-0.388520	0.169158	0.123085	-0.148234	-0.922332	-0.082432	-0.304487	-0.634184	-0.563015	-0.479181	...	-1.

5 rows × 22 columns

Question

Why is it import to use the same μ and σ to perform normalization on the training and validation data? What would happen if we use μ and σ estimated using the validation to perform normalization on the validation data?

Using the same mean () and standard deviation () for normalizing both the training and validation data is crucial because it ensures that the model is trained and evaluated on a consistent scale, allowing the model to learn and be tested properly. If we normalize the validation data using its own mean and standard deviation, the model could unintentionally pick up information from that data during training. This can make the model appear to perform better than it really does, leading to inaccurate predictions when it faces new data in the real world.

Part 1 (15 pts) Concrete closed form solution

Part 1 (15 pts) Generate closed-form solution for reference.

Our data now contains 21 numeric features, before we learn a linear regression model using gradient descent, we will first build the closed-form solution as a reference point. So for this part, you need to

1. Implement the close-form solution for linear regression and apply it to the training data to learn the weight vector for your linear regression model. For matrix inversion you can use existing numpy functions. Specifically, we recommend the `numpy.linalg.pinv()` function for inverting near-singular matrices.
2. Apply your learned linear regression model to the training data to make predictions for all training examples and report the Mean Squared Error.
3. Apply your learned linear regression model to the validation data to make predictions for all the validation examples and report the mean squared error for the validation data.

Your code should print the weight vector, which has 22 dimensions, one for each feature plus one additional w_0 . Your code should also report the MSE for the training and validation data respectively.

```
In [76]: features = train_df.columns.difference(['price'])

X_train = np.c_[np.ones(train_df.shape[0]), train_df[features]]
y_train = train_df['price'].values
X_val = np.c_[np.ones(val_df.shape[0]), val_df[features]]
y_val = val_df['price'].values

w = np.linalg.pinv(X_train.T @ X_train) @ X_train.T @ y_train

train_pred = X_train @ w
train_mse = np.mean((y_train - train_pred) ** 2)
print("Training MSE:", train_mse)

val_pred = X_val @ w
val_mse = np.mean((y_val - val_pred) ** 2)
print("Validation MSE:", val_mse)

weight_vector = pd.Series(w, index=['w0'] + list(features))
display(weight_vector)

print("Length of weight vector (w):", len(w))
print("Length of index list (including 'w0'):", len(['w0'] + list(features)))

Training MSE: 3.757887089954586
Validation MSE: 4.5035081053568575
```

	0
w0	5.361673
age_since_renovated	-0.102558
bathrooms	0.339072
bedrooms	-0.281353
condition	0.199843
day	-0.050637
floors	0.018137
grade	1.115443
lat	0.836612
long	-0.303696
month	0.054850
sqft_above	0.756233
sqft_basement	0.155462
sqft_living	0.763420
sqft_living15	0.143581
sqft_lot	0.058150
sqft_lot15	-0.099274
view	0.446754
waterfront	0.328139
year	0.173750
yr_built	-0.883362
zipcode	-0.263419

dtype: float64

Length of weight vector (w): 22

Length of index list (including 'w0'): 22

Question

The learned feature weights are often used to understand the importance of the features. The sign of the weights indicates if a feature positively or negatively impact the price, and the magnitude suggests the strength of the impact. Does the sign of all the features match your expectation based on your common-sense understanding of what makes a house expensive? Please highlight any surprises from the results.

Most feature weights align with our expectations regarding house pricing. Positive impact features like bathrooms, grade, lat, view, sqft_living, and waterfront are making the house expensive. These make intuitive sense, as buyers often value more bathrooms, higher quality (grade), scenic views, larger living spaces, and waterfront access.

But on the other hand, some features, like the year built and zipcode, have negative weights, indicating they tend to lower the price. This could mean that older homes or certain areas might be less desirable, which is also reasonable in many cases.

However, there are some surprises in the data. For instance, bedrooms have a negative weight, suggesting that simply adding more bedrooms doesn't necessarily increase a home's value in this dataset. This might imply that buyers are prioritizing other factors over just the number of bedrooms. Another unexpected finding is the negative weight on `sqft_lot15`. With a weight of -0.099, it implies that larger lot sizes don't always add value, which goes against the common belief that more land should increase a property's value. This could be due to neighborhood characteristics or the way lot size interacts with other features in the data.

Part 2 (40 pts) Implement and experiment with batch gradient descent

Your implementation should take following inputs:

1. the training data (with d features and 1 target variable y),
2. the learning rate γ ,

- the number of iterations $\$T\$$
- Optional convergence threshold (optional) $\$\epsilon_l\$$ for the loss or $\$\epsilon_g\$$ for the norm of the gradient

It should output:

- the learned $\$d+1\$$ - dimensional weight vector
- the sequence of $\$T\$$ MSE losses, one for each training epoch. You will be asked to plot the losses as a function of training epoch later.

```
In [77]: def batch_gradient_descent(X, y, learning_rate, num_iterations, epsilon=None, convergence_threshold=None):
    X = np.c_[np.ones(X.shape[0]), X]

    weights = np.random.randn(X.shape[1])
    losses = []

    for _ in range(num_iterations):
        gradients = 2 / len(X) * X.T.dot(X.dot(weights) - y)
        weights -= learning_rate * gradients

        mse = np.mean((X.dot(weights) - y) ** 2)
        losses.append(mse)

        if (epsilon and mse < epsilon) or (convergence_threshold and np.linalg.norm(gradients) < convergence_th
            break

    return weights, losses

#example
batch_gradient_descent(
    train_df[features],
    y_train,
    learning_rate=0.05,
    num_iterations=1000,
    epsilon=1e-4,
    convergence_threshold=1e-4
)
```

```
Out[77]: (array([ 5.36167284, -0.10217076,  0.33907827, -0.28134638,  0.19982005,
        -0.05064146,  0.01814189,  1.11546834,  0.83660836, -0.3036956 ,
         0.05479668,  0.25830447, -0.10601228,  1.30990867,  0.1435462 ,
         0.05811468, -0.09923711,  0.44676535,  0.32814223,  0.1736877 ,
        -0.88299302, -0.26341713]),
 [57.579327873647635,
  44.05651309857181,
  34.80276367439203,
  28.06577475194115,
  23.008920357186692,
  19.138559724408996,
  16.13294175452952,
  13.77190845336726,
  11.900101090182698,
  10.405175930693622,
  9.204161330391207,
  8.234641461641482,
  7.448919102535408,
  6.810061294202792,
  6.289158773833718,
  5.863383558760532,
  5.51458248941883,
  5.22823861483329,
  4.992690669420886,
  4.798537567933039,
  4.638178234856219,
  4.505452250489709,
  4.395356813526945,
  4.303822270858995,
  4.227533111083625,
  4.163784583590862,
  4.110367447953389,
  4.065475072097797,
  4.027628373495675,
  3.9956150621418054,
  3.9684403830753725,
  3.9452871287624074,
  3.9254831394028606,
  3.908474862023623,
  3.8938058189125813,
  3.881099058783603,
  3.8700428423023303,
  3.860378956627387,
  3.851893168683725,
  3.8444074196441975,
  3.8377734380021504,
  3.831867509193,
```

3.8265861887655306,
3.821842785849389,
3.8175644759038896,
3.813689927902493,
3.810167352364242,
3.8069528939208097,
3.8040093061588096,
3.801304857910389,
3.7988124294735126,
3.796508764825677,
3.7943738520742882,
3.7923904094259147,
3.7905434580676554,
3.7888199667098084,
3.787208555279988,
3.7856992474989686,
3.7842832639003645,
3.7829528483551904,
3.781701122389741,
3.780521962590899,
3.7794098972176404,
3.7783600188142374,
3.777367910176432,
3.776429581478668,
3.7755414167463277,
3.774700128166336,
3.7739027169846397,
3.7731464399494663,
3.7724287804331236,
3.7717474235088195,
3.771100234377958,
3.7704852396419493,
3.7699006109944277,
3.7693446509777337,
3.768815780504061,
3.7683125278888316,
3.767833519183131,
3.7673774696249196,
3.7669431760562064,
3.7665295101764102,
3.7661354125214808,
3.765759887074603,
3.7654019964280327,
3.7650608574271027,
3.764735637237307,
3.764425549783533,
3.7641298525176055,
3.7638478434762317,
3.763578858596521,
3.7633222692605766,
3.763077480044377,
3.762843926649308,
3.762621073997442,
3.762408414474002,
3.7622054663024445,
3.7620117720393593,
3.7618268971778646,
3.761650428849492,
3.76148197461569,
3.7613211613410487,
3.761167634141223,
3.761021055399271,
3.760881103844781,
3.760747473690752,
3.760619873823677,
3.760498027042766,
3.7603816693445675,
3.760270549249696,
3.760164427168591,
3.760063074803565,
3.7599662745846207,
3.759873819136744,
3.7597855107765663,
3.7597011610364834,
3.7596205902144413,
3.7595436269477913,
3.7594701078096886,
3.7593998769266697,
3.759332785616114,
3.759268692042426,
3.759207460890816,
3.7591489630576698,
3.7590930753565654,
3.7590396802390327,
3.7589886655292597,
3.7589399241719392,
3.758893353992574,
3.758848857469539,
3.7588063415172837,

3.758765717280072,
3.7587268999357235,
3.758689808508808,
3.7586543656928195,
3.7586204976808704,
3.7585881340044542,
3.7585572073798823,
3.7585276535619974,
3.7584994112048107,
3.7584724217286993,
3.7584466291938585,
3.7584219801796794,
3.7583984236697843,
3.758375910942421,
3.758354395465968,
3.758333832799301,
3.7583141804967792,
3.758295398017638,
3.7582774466395747,
3.7582602893763197,
3.758243890899009,
3.7582282174611805,
3.758213236827218,
3.7581989182040725,
3.7581852321761287,
3.758172150643041,
3.7581596467604217,
3.758147694883235,
3.7581362705117756,
3.7581253502401126,
3.7581149117068797,
3.7581049335483048,
3.758095395353379,
3.7580862776210537,
3.758077561719391,
3.758069229846561,
3.758061264993606,
3.7580536509088915,
3.758046372064175,
3.7580394136221877,
3.758032761405707,
3.7580264018680074,
3.7580203220646533,
3.758014509626558,
3.7580089527342686,
3.758003640093396,
3.757998560911161,
3.75799370487399,
3.757989062126123,
3.7579846232491825,
3.757980379242656,
3.7579763215052626,
3.7579724418171505,
3.7579687323228956,
3.757965185515264,
3.7579617942197014,
3.7579585515795135,
3.7579554510417204,
3.757952486343535,
3.757949651499452,
3.7579469407889103,
3.7579443487445157,
3.7579418701407805,
3.7579394999833697,
3.7579372334988346,
3.7579350661247948,
3.757932993500563,
3.7579310114581923,
3.7579291160139174,
3.7579273033599785,
3.757925569856813,
3.7579239120255936,
3.757922326541101,
3.757920810224904,
3.7579193600388643,
3.7579179730789063,
3.757916646569086,
3.7579153778559062,
3.757914164402898,
3.757913003785434,
3.75791189368578,
3.7579108318883603,
3.7579098162752382,
3.7579088448217943,
3.7579079155925963,
3.7579070267374584,
3.7579061764876673,
3.757905363152382,
3.757904585115191,

3.757903840830822,
3.7579031288219995,
3.7579024476764387,
3.757901796043975,
3.757901172633825,
3.7579005762119557,
3.757900005598588,
3.7578994596657957,
3.7578989373352205,
3.7578984375758817,
3.757897959402091,
3.757897501871449,
3.757897064082941,
3.7578966451751077,
3.7578962443243062,
3.7578958607430395,
3.7578954936783644,
3.75789514241037,
3.757894806250723,
3.7578944845412736,
3.7578941766527287,
3.7578938819833807,
3.7578935999578964,
3.7578933300261474,
3.7578930716621115,
3.7578928243628043,
3.757892587647269,
3.7578923610556068,
3.7578921441480486,
3.757891936504076,
3.7578917377215686,
3.7578915474159973,
3.757891365219652,
3.7578911907809025,
3.757891023763492,
3.757890863845858,
3.7578907107204955,
3.757890564093331,
3.7578904236831367,
3.7578902892209665,
3.7578901604496178,
3.7578900371231128,
3.7578899190062107,
3.7578898058739316,
3.7578896975111102,
3.7578895937119654,
3.757889494279687,
3.7578893990260407,
3.7578893077709994,
3.757889220342374,
3.75788913657548,
3.7578890563127967,
3.757888979403667,
3.757888905703986,
3.75788883507592,
3.75788876738763,
3.757888702513007,
3.75788864033143,
3.7578885807275113,
3.757888523590884,
3.7578884688159686,
3.757888416301772,
3.757888365951684,
3.7578883176732862,
3.7578882713781683,
3.757888226981754,
3.757888184403133,
3.7578881435649003,
3.7578881043930035,
3.7578880668165975,
3.7578880307679023,
3.757887996182072,
3.757887962997064,
3.7578879311535194,
3.7578879005946417,
3.757887871266093,
3.7578878431158755,
3.757887816094241,
3.7578877901535854,
3.757887765248357,
3.7578877413349683,
3.757887718371711,
3.7578876963186714,
3.7578876751376544,
3.7578876547921083,
3.7578876352470516,
3.7578876164690103,
3.7578875984259454,
3.7578875810871932,

3.757887564423407,
3.757887548406498,
3.7578875330095833,
3.757887518206929,
3.757887503973906,
3.757887490286934,
3.7578874771234463,
3.757887464461839,
3.7578874522814276,
3.7578874405624174,
3.757887429285849,
3.7578874184335778,
3.7578874079882296,
3.7578873979331684,
3.7578873882524686,
3.757887378930879,
3.7578873699537967,
3.7578873613072377,
3.757887352977812,
3.7578873449526955,
3.7578873372196075,
3.7578873297667865,
3.7578873225829685,
3.757887315657362,
3.7578873089796345,
3.7578873025398862,
3.7578872963286343,
3.7578872903367966,
3.7578872845556677,
3.757887278976912,
3.7578872735925426,
3.757887268394904,
3.7578872633766633,
3.7578872585307934,
3.7578872538505603,
3.7578872493295097,
3.7578872449614567,
3.7578872407404744,
3.7578872366608786,
3.757887232717224,
3.7578872289042904,
3.7578872252170696,
3.757887221650766,
3.757887218200777,
3.7578872148626927,
3.7578872116322803,
3.757887208505484,
3.757887205478413,
3.757887202547336,
3.7578871997086742,
3.7578871969589924,
3.757887194294998,
3.757887191713531,
3.7578871892115595,
3.7578871867861734,
3.7578871844345816,
3.7578871821541044,
3.75788717994217,
3.7578871777963103,
3.757887175714156,
3.7578871736934314,
3.7578871717319533,
3.7578871698276233,
3.7578871679784283,
3.7578871661824333,
3.7578871644377796,
3.7578871627426844,
3.7578871610954323,
3.757887159494376,
3.7578871579379314,
3.7578871564245806,
3.7578871549528574,
3.7578871535213603,
3.7578871521287356,
3.7578871507736857,
3.757887149454962,
3.7578871481713625,
3.757887146921732,
3.7578871457049607,
3.7578871445199793,
3.757887143365757,
3.7578871422413083,
3.757887141145677,
3.7578871400779486,
3.75788713903724,
3.757887138022701,
3.757887137033515,
3.757887136068892,
3.7578871351280743,

```

3.757887134210331,
3.757887133314956,
3.7578871324412733,
3.7578871315886264,
3.7578871307563864,
3.7578871299439434,
3.757887129150713,
3.757887128376129,
3.7578871276196466,
3.7578871268807394,
3.7578871261589,
3.7578871254536392,
3.7578871247644847,
3.7578871240909786,
3.7578871234326816,
3.757887122789169,
3.7578871221600276,
3.7578871215448624,
3.7578871209432907,
3.7578871203549395,
3.7578871197794532,
3.757887119216484,
3.757887118665698,
3.7578871181267712,
3.75788711759939,
3.757887117083253,
3.7578871165780665,
3.7578871160835474])

```

You will now experiment with the batch gradient descent algorithm with different learning rate on the provided data.

Please train your model for up to 3000 iterations using different learning rate: $\gamma = 10^{-i}$, $i=0,1,\dots,4$. For each learning rate, you can opt to stop early if it has converged (using the convergence threshold) or diverged (the loss start to grow larger).

For each converging (not necessarily converged yet) learning rate, please compute and report the MSE of the final learned weights on the validation data.

Also please provide a plot that shows the training loss curves (MSE as a function of the # of epochs) for all the converging learning rates. Please use different colors mark different learning rates and provide proper legends for your figure.

```

In [78]: learning_rates = [10**-i for i in range(5)]
num_iterations = 3000
convergence_threshold = 1e-6
epsilon = 1e-8

weights_list = []
validation_mses = []
all_losses = []

for lr in learning_rates:
    weights, losses = batch_gradient_descent(train_df[features], y_train, lr, num_iterations, epsilon, convergence_threshold)
    weights_list.append(weights)
    all_losses.append(losses)

    X_val_b = np.c_[np.ones(val_df.shape[0]), val_df[features]]
    y_val_pred = X_val_b.dot(weights)
    mse_val = np.mean((y_val_pred - y_val) ** 2)
    validation_mses.append(mse_val)

plt.figure(figsize=(10, 6))
for i, losses in enumerate(all_losses):
    if learning_rates[i] == 1:
        continue
    plt.plot(losses, label=f'Learning rate: {learning_rates[i]}')
plt.xlabel('Epochs')
plt.ylabel('Mean Squared Error (MSE)')
plt.title('Training Loss Curves for Different Learning Rates')
plt.legend()
plt.show()

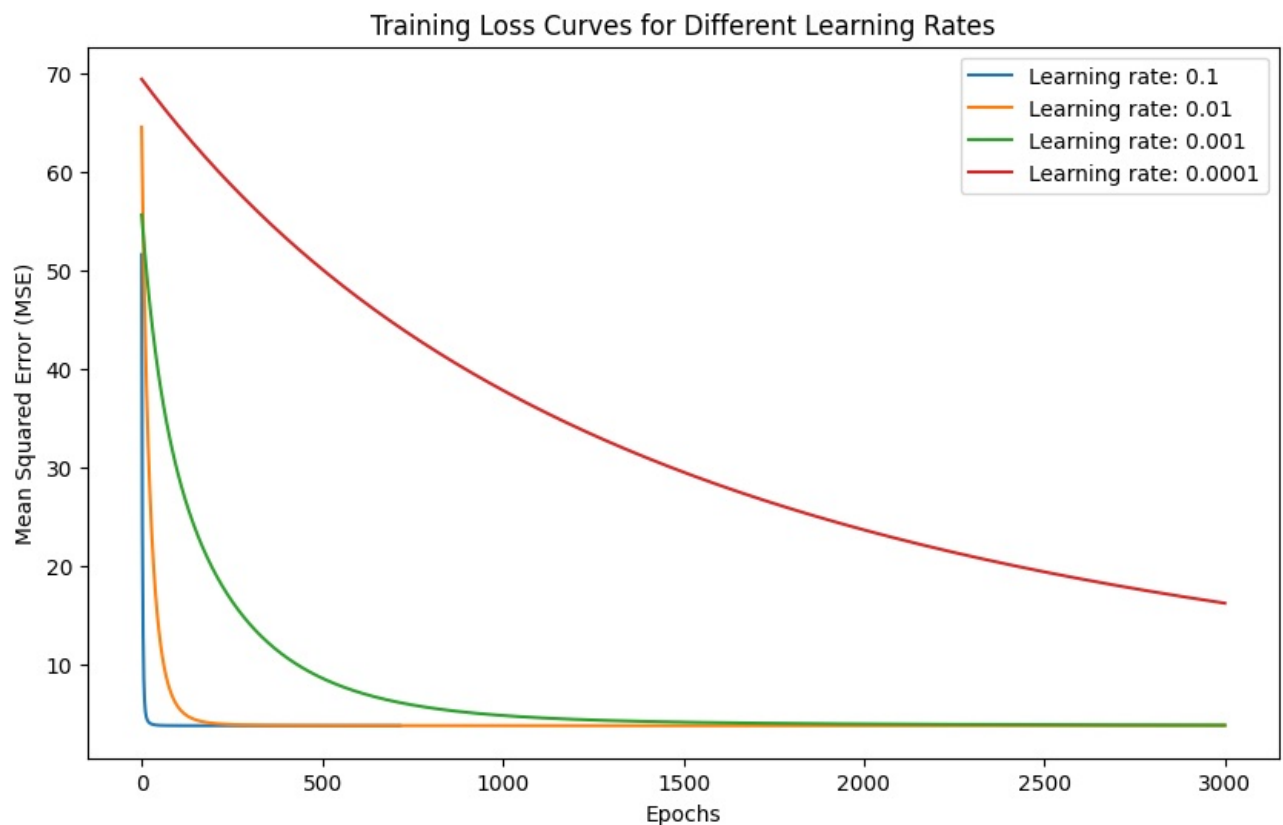
for lr, mse in zip(learning_rates, validation_mses):
    print(f"Learning rate: {lr}, Validation MSE: {mse}")

```

```

/usr/local/lib/python3.10/dist-packages/numpy/core/_methods.py:118: RuntimeWarning: overflow encountered in reduce
    ret = umr_sum(arr, axis, dtype, out, keepdims, where=where)
<ipython-input-77-4e299f36a189>:11: RuntimeWarning: overflow encountered in square
    mse = np.mean((X.dot(weights) - y) ** 2)

```



Learning rate: 1, Validation MSE: nan
 Learning rate: 0.1, Validation MSE: 4.503508090163732
 Learning rate: 0.01, Validation MSE: 4.503498111507704
 Learning rate: 0.001, Validation MSE: 4.539809995982212
 Learning rate: 0.0001, Validation MSE: 16.615115268273815

Question

Which learning rate leads to the best training and validation MSE respectively? Do you observe better training MSE tend to correspond to better validation MSE? How is this different from the trend shown on page 51 of the lecture slides (titled danger of using training loss to select M) regarding overfitting? Is there any issue with using training loss to pick learning rate in this case?

1. The best learning rate is the one that gives the lowest validation MSE which is 0.01 in our case, as that shows your model is performing well on new, unseen data. While a lower training MSE is usually a good sign, it's important to see if it aligns with a low validation MSE.
2. In an ideal scenario, a lower training MSE should generally correspond to a lower validation MSE, showing that the model is learning well from the data and generalizing properly, but if the training MSE is much lower than the validation MSE, this is often bad for overfitting, meaning the model is not performing well on data it hasn't seen before.
3. As the model complexity () increases, training loss keeps going down, suggesting the model is improving. However, validation loss the measure of how well the model performs on new data—initially drops and then starts to rise, which signals overfitting. This means the model is fitting the training data well but isn't generalizing to new data. The same issue can occur when picking a learning rate. A rate that minimizes training MSE doesn't always minimize validation MSE. If the training MSE is much lower than the validation MSE, it likely indicates overfitting, where the model memorizes the training data but performs poorly on unseen data. To avoid this, it's important to look at both training and validation MSE. This way, you can choose a learning rate that balances good training performance with strong generalization to new data, reducing the risk of overfitting.
4. Relying solely on training loss to pick a learning rate can be risky as you could end up with a learning rate that allows the model to overfit the training data, resulting in poor performance on validation data. By considering both training and validation MSE, you're more likely to find a learning rate that balances effective learning with good generalization, which helps avoid overfitting.

Part 3. More exploration.

3(a). (20 pts) Normalization of features: what is the impact?

In part 1, you were asked to perform z-score normalization of all the features. In this part, we will ask you to first conceptually think about what is the impact this operation on the solution and then use some experiments to verify your conceptual understanding.

Questions.

The normalization process applies a linear transformation to each feature, where the transformed feature x' is simply a linear function of original feature x : $x' = \frac{x - \mu}{\sigma}$.

Let's dissect the influence of this transformation on our learned linear regression model.

1. How do you think this transformation will influence the training and validation MSE we get for the closed-form solution? Why?
 2. How do you think this will change the magnitude of the weights of the learned model? Why?
 3. How do you think this will change the convergence behavior of the batch gradient descent algorithm? Why?
-
1. By normalizing the features, we make sure that all the features contribute equally to the model. This helps the model learn better patterns in the data and reduce overfitting. Since all features are transformed to have a mean of 0 and a standard deviation of 1, the model can find patterns more easily. This often results in a lower Mean Squared Error (MSE) during both training and validation because the predictions are more aligned with the actual target values. Ultimately this leads to better performance on both training and validation datasets
 2. Normalising the features will affect the size of the weights in the trained model because it adjusts the features to a consistent range, usually centered around a mean of 0 with a standard deviation of 1. This adjustment enables the model to concentrate on the connections between the features and the target variable without being swayed by the original scales of the features. As a result, the learned weights may become smaller in magnitude, reflecting the relative importance of each feature in a more balanced way. This process helps ensure that features with larger ranges do not influence the model's predictions.
 3. Normalization can speed up the learning process because the model's updates are more stable. When features are on the same scale, the algorithm can take larger steps toward finding the best weights without overshooting. With normalized data, the gradient descent algorithm is less likely to bounce around and more likely to make steady progress toward the solution, leading to smoother learning.

Now please perform the following experiments to verify your answer to the above questions.

1. Apply closed-form solution to data that did not go through the feature normalization step, and report the learned weights and the resulting training and testing MSEs.
2. Apply gradient descent algorithm to data that did not go through the feature normalization step using different learning rate. Note that the learning rate used in previous section will no longer work here. You will need to search for an appropriate learning rate to get some converging behavior. Plot your MSE loss curve as a function of the epochs once you identify a convergent learning rate. Hint: the learning rate needs to be much, much, much, much, much smaller (think about each much as an order of magnitude) than what was used in part 2). Also unless you let it run for a long time, it is unlikely to converge to the same level of loss values. So use an upper bound on the # of iterations so that it won't take forever.

In [79]: *#Applying closed form solution to non-normalised data*

```
raw_train_data = pd.read_csv(train_path)
raw_val_data = pd.read_csv(val_path)
```

```
#1. drop the ID column from training and validation data
unnormal_train_df = raw_train_data.drop(columns=['id'])
unnormal_val_df = raw_val_data.drop(columns=['id'])
```

```
#2. change date into 3 numerical features day, month and year, like in the warm up exercise
```

```

for df in [unnormal_train_df, unnormal_val_df]:
    df['date'] = pd.to_datetime(df['date'], format='%m/%d/%Y')
    #extract month, day, and year into separate columns
    df['month'] = df['date'].dt.month
    df['day'] = df['date'].dt.day
    df['year'] = df['date'].dt.year
    #drop the original date column
    df = df.drop(columns=['date'], inplace=True)

#3. Adding column age_since_renovated and replacing "yr_renovated"

def calculate_building_age(row):
    if row['yr_renovated'] != 0:
        return row['year'] - row['yr_renovated']
    else:
        return row['year'] - row['yr_built']

for df in [unnormal_train_df, unnormal_val_df]:
    df['age_since_renovated'] = df.apply(calculate_building_age, axis=1)
    df.drop(columns=['yr_renovated'], inplace=True)

features = unnormal_train_df.columns.difference(['price'])

X_train = np.c_[np.ones(unnormal_train_df.shape[0]), unnormal_train_df[features]]
y_train = unnormal_train_df['price'].values
X_val = np.c_[np.ones(unnormal_val_df.shape[0]), unnormal_val_df[features]]
y_val = unnormal_val_df['price'].values
w = np.linalg.pinv(X_train.T @ X_train) @ X_train.T @ y_train
print("Weights\n")
print(f"Feature: intercept, Weight: {w[0]}") # Add 'intercept' as the first feature

for feature, weight in zip(features, w[1:]):
    print(f"Feature: {feature}, Weight: {weight}")

train_pred = X_train @ w

train_mse = np.mean((y_train - train_pred) ** 2)
print("\nMSE for Training data:", train_mse)
val_pred = X_val @ w
val_mse = np.mean((y_val - val_pred) ** 2)
print("\nMSE for Validation data:", val_mse)

def batch_gradient_descent(X, y, learning_rate, num_iterations, epsilon=None, convergence_threshold=None):
    # Add a column of ones to include the intercept term
    X_b = np.c_[np.ones((X.shape[0],1)), X] # shape (n_samples, d+1)

    # Initialize weights
    np.random.seed(0)
    weights = np.random.randn(X_b.shape[1])

    losses = [] # List to store the loss at each iteration

    for iteration in range(num_iterations):
        gradients = 2/len(X_b) * X_b.T.dot(X_b.dot(weights) - y)
        weights = weights - learning_rate * gradients

        # MSE calculation
        y_pred = X_b.dot(weights)
        mse = np.mean((y_pred - y) ** 2)
        losses.append(mse)

        # checking convergence
        if epsilon is not None:
            if mse < epsilon:
                break
        if convergence_threshold is not None:
            if np.linalg.norm(gradients) < convergence_threshold:
                break

    return weights, losses

learning_rates = [9.8**-i for i in range(14,10,-1)]
num_iterations = 1000
convergence_threshold = 1e-6
epsilon = 1e-8

#Experiment with different learning_rates
final_weights = []
validation_mse = []
losses = []

for rate in learning_rates :

```

```

weights, loss = batch_gradient_descent(unnormal_train_df[features], y_train, rate, num_iterations, epsilon, c)
final_weights.append(weights)
losses.append(loss)

#Compute MSE on validation data
X_val_b = np.c_[np.ones(unnormal_val_df.shape[0]), unnormal_val_df[features]]

y_val_pred = X_val_b.dot(weights)

mse_val = np.mean((y_val_pred - y_val ) ** 2)
validation_mse.append(mse_val)

# plot the training losses
plt.figure(figsize=(10,6))
for i, loss in enumerate(losses):
    if learning_rates[i]==1:
        continue
    plt.plot(range(len(loss)),loss,label=f"Learning rate :{learning_rates[i]}")

plt.xlabel('Number of Epochs')
plt.ylabel('Mean Squared Error (MSE)')
plt.title('Training Loss curves for different learning rates')
plt.legend()
plt.show()

for i, mse in enumerate(validation_mse):
    print(f"Learning rate:{learning_rates[i]}, validation MSE: {mse}")

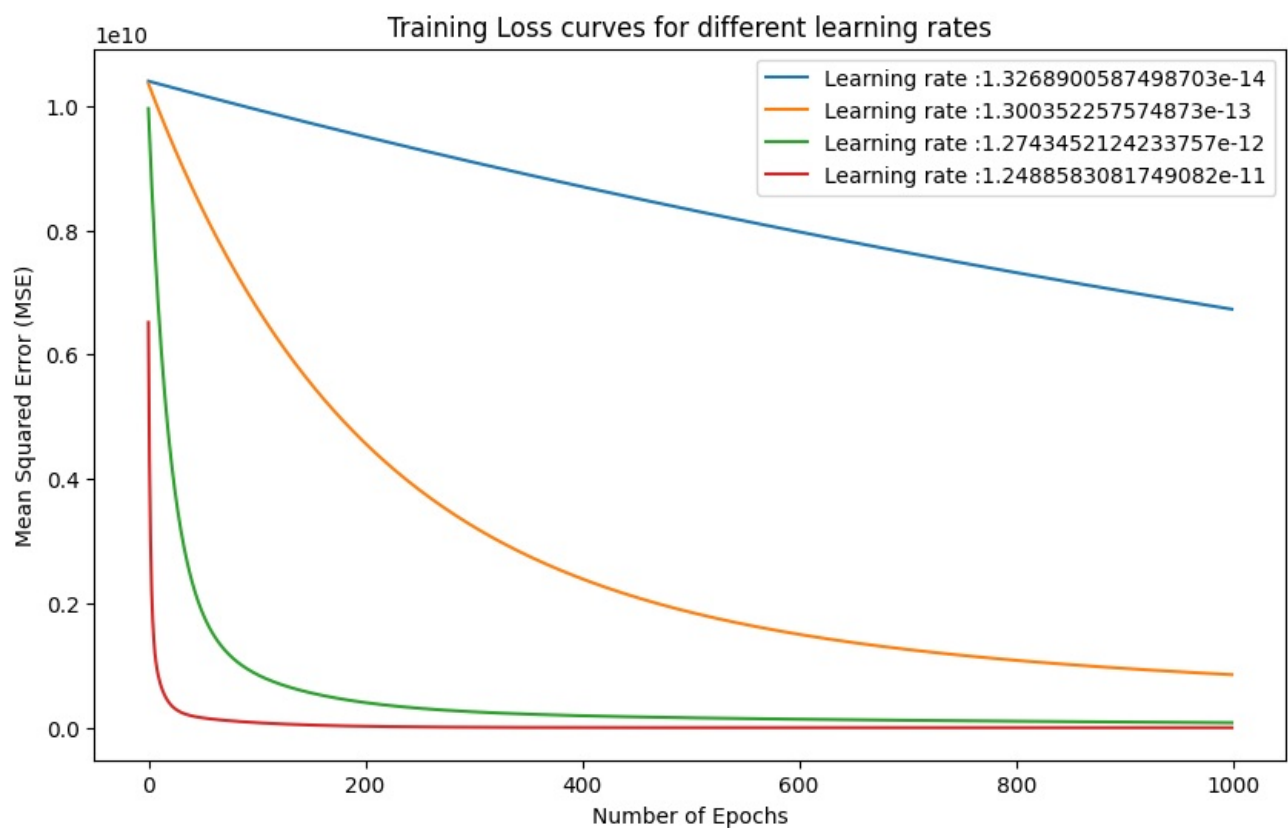
```

Weights

Feature: intercept, Weight: 0.0001808332051459062
 Feature: age_since_renovated, Weight: -0.0031683550200548275
 Feature: bathrooms, Weight: 0.44004184947541874
 Feature: bedrooms, Weight: -0.29494358813734634
 Feature: condition, Weight: 0.2907117793248176
 Feature: day, Weight: -0.006969491176609143
 Feature: floors, Weight: 0.04483023323982918
 Feature: grade, Weight: 0.951607363923924
 Feature: lat, Weight: 6.0087155245200075
 Feature: long, Weight: -2.135938805463776
 Feature: month, Weight: -0.023201459599702504
 Feature: sqft_above, Weight: 0.0007635994619053514
 Feature: sqft_basement, Weight: 0.00022068177896200885
 Feature: sqft_living, Weight: 0.0009842732372912447
 Feature: sqft_living15, Weight: 0.00019448131797468582
 Feature: sqft_lot, Weight: 1.3494867921265564e-06
 Feature: sqft_lot15, Weight: -3.4334744834654583e-06
 Feature: view, Weight: 0.5987603181555551
 Feature: waterfront, Weight: 4.017621503759493
 Feature: year, Weight: 0.0244379978471295
 Feature: yr_built, Weight: -0.030127428189929487
 Feature: zipcode, Weight: -0.005538373230356086

MSE for Training data: 3.7690052852156177

MSE for Validation data: 4.516144824902658



Learning rate:1.3268900587498703e-14, validation MSE: 5800919220.233118
 Learning rate:1.300352257574873e-13, validation MSE: 527718080.2864878
 Learning rate:1.2743452124233757e-12, validation MSE: 68158480.39460975
 Learning rate:1.2488583081749082e-11, validation MSE: 2015151.7282523513

Questions

Please revisit the questions above. Does your experiment confirm your expectation? Can you provide explanations to the observed differences (or lack of differences) between the normalized data and unnormalized data? Based on these observations and your understanding of them, please comment on the benefits of normalizing the input features in learning for linear regressions.

Yes, The experiment confirmed my expectations.

The weights for unnormalized data are quite small and can even be close to zero for some features, such as the intercept and `sqft_lot`. This means the model isn't doing a good job of understanding how these features relate to the target variable. In contrast, features with bigger values, like `lat` and

`waterfront`, have much larger weights, showing that they have a stronger influence on the predictions. Normalization reduces the risk that features with larger value ranges dominate the model's predictions allowing for a more impartial representation of all features.

Also MSE for normalised train and test data is smaller as compared to unnormalised data, which means the model predictions are closer to the target values on average.

Benifits of normalizing the input features in linear regression are:

Improved Model Performance: When features are on a similar scale, the model can learn better, leading to more accurate predictions.

Faster Convergence: Normalized data helps the algorithm reach the best solution faster during training. This means the model will learn quicker and take fewer iterations to improve.

Stability: Normalized data reduces the risk of numerical issues during calculations, making the training process more stable.

3(b). (20 pts) Explore the impact of correlated features

In the warm up exercise, you all have seen some features are highly correlated with one another. For example, there are multiple squared footage related features that are strongly correlated (e.g., `sqft_above` and `sqft_living` has a correlation coefficient of 0.878). This is referred to as multicollinearity phenomeon, where two or more features are correlated.

There are numerous consequences from multicollinearity. It makes it more challenging to estimate the weights of the features accurately. The weights may become unstable, and their interpretation becomes less clear.

In this part you will work with the pre-processed training set, and perform the following experiments **using the closed-form solution**. Specifically, your code should:

1. Create five slightly different training sets, each of which is obtained by randomly subsample 75% of the orginial training set.
2. Use the closed-form solution of linear regression to fit the model on each of the five training sets.
3. For each model, report the learned weight vector in a table. The table should have five rows (one for each model) and a column for each feature's weight. Include a header row to clearly label the feature names for each column.
4. Compute the variance of the learned weight coefficients across the five models for each feature. This variance will serve as a measure of the **stability** of the weight assigned to each feature.

```
In [80]: # Number of subsamples to generate
num_subsamples = 5
subsample_fraction = 0.75

# List to store weight vectors from each subsample
weights_list = []

# Create five subsamples and calculate weights for each using closed-form solution
for i in range(num_subsamples):
    # Randomly sample 75% of the training data without replacement
    subsample_df = train_df.sample(frac=subsample_fraction, random_state=i)

    # Prepare X and y for the subsample
    X_sub = np.c_[np.ones(subsample_df.shape[0]), subsample_df[features]]
    y_sub = subsample_df['price'].values

    # Closed-form solution on the subsample
    weights_sub = np.linalg.pinv(X_sub.T @ X_sub) @ X_sub.T @ y_sub
    weights_list.append(weights_sub)

# Convert the list of weight vectors to a DataFrame for easy analysis
weights_df = pd.DataFrame(weights_list, columns=['Intercept'] + features.tolist())
print("Weight Vectors from Different Subsamples:\n", weights_df)

# Calculate the variance of the weights across the five models for each feature
weight_variances = weights_df.var()
print("\nVariance of Weights Across Subsamples:\n", weight_variances)
```

Weight Vectors from Different Subsamples:

	Intercept	age_since_renovated	bathrooms	bedrooms	condition	day	\
0	5.362070	-0.154176	0.338113	-0.280649	0.170545	-0.049773	
1	5.359410	-0.152128	0.265747	-0.230422	0.198973	-0.048657	
2	5.363136	-0.011776	0.353731	-0.271090	0.175726	-0.034395	
3	5.365566	-0.151848	0.240452	-0.310284	0.225093	-0.066759	
4	5.346457	-0.119904	0.348320	-0.259355	0.191296	-0.049236	

	floors	grade	lat	long	...	sqft_basement	sqft_living	\
0	-0.018366	1.124965	0.846749	-0.276303	...	0.149856	0.818178	
1	0.062894	1.234219	0.812042	-0.274539	...	0.185751	0.675581	
2	0.025311	1.125696	0.836203	-0.335214	...	0.140682	0.746785	
3	0.082098	1.144139	0.828441	-0.283588	...	0.194643	0.733742	
4	-0.028118	1.053304	0.848799	-0.323829	...	0.112256	0.801248	

	sqft_living15	sqft_lot	sqft_lot15	view	waterfront	year	\
0	0.049129	0.034165	-0.087401	0.429073	0.348830	0.149650	
1	0.180668	0.062027	-0.079587	0.367137	0.389410	0.176136	
2	0.176401	0.061608	-0.076462	0.483437	0.229114	0.211192	
3	0.207019	0.062896	-0.092200	0.445641	0.317673	0.180232	
4	0.086751	0.063563	-0.111034	0.507086	0.326710	0.169678	

	yr_built	zipcode
0	-0.945102	-0.258585
1	-0.936367	-0.249398
2	-0.815799	-0.269341
3	-0.903080	-0.284776
4	-0.851625	-0.267515

[5 rows x 22 columns]

Variance of Weights Across Subsamples:

Intercept	0.000057
age_since_renovated	0.003727
bathrooms	0.002741
bedrooms	0.000854
condition	0.000467
day	0.000132
floors	0.002350
grade	0.004195
lat	0.000225
long	0.000820
month	0.000248
sqft_above	0.005793
sqft_basement	0.001141
sqft_living	0.003239
sqft_living15	0.004641
sqft_lot	0.000161
sqft_lot15	0.000186
view	0.002909
waterfront	0.003483
year	0.000495
yr_built	0.003084
zipcode	0.000174

dtype: float64

Questions

Ideally, we would like the weight coefficients to be stable across different runs, as this increases confidence in the model's reliability. Do highly correlated features tend to exhibit more instability in their weights across different training sets compared to less correlated features? Discuss any trend you observe based on the variance of the weight coefficients. How does the stability of these features relate to the multicollinearity issue present in this dataset?

Yes, highly correlated features do tend to show more instability in their weight coefficients when we run the model on different training sets.

For example, the features `sqft_living` and `sqft_above` are likely to be positively correlated. Homes with larger living areas generally have more square footage above ground.

However in one run, `sqft_living` could have a weight of 0.82, while `sqft_above` might show a weight of 0.65. In another run, `sqft_living` could drop to 0.45, and `sqft_above` could rise to 0.72.

The feature `year_built` is less likely to be correlated with other features like `waterfront`. If we examine the weight coefficient for `year_built`, we might find it consistently around 0.10 across different training runs.

This happens because, with multicollinearity, it's difficult for the model to decide which of the correlated features should get more weight. As a result, the importance can shift between these features across different runs, leading to inconsistent weights.

Bonus. In-class competition (5 bonus pts)

We will host a in-class competition using the IA1 data, where you are encouraged to explore different ways to improve the prediction performance by manipulating the data. This could include: feature engineering such as removing, transforming features, constructing new features based on existing ones, using different encoding for the discrete features; data manipulation such as identifying and removing potential outliers; and target manipulation such as log transforming the price target. This is where you can get creative and test your ideas out.

To participate in this competition, use the following link:

<https://www.kaggle.com/t/7a885211273e48968e3a5f1b556cb685>

You should continue working in the same team for this competition. The training and validation data provided on the kaggle site are the same as the IA1 assignment. To participate, you will need to train your model and apply it to testing data provided on kaggle, and submit prediction files to be scored.

Your scoring will have two parts, the performance on the public leader board as well as the private leader board. The results on the public leader board is visible through out the competition so that you can gauge how well your model is performing in comparison to others. The private leader board shows the final evaluation performance and will be released only once after the competition is closed.

Each team will be allowed to submit 3 final entries to be evaluated. You can use the public leaderboard performance to pick which models to use for your final evaluation entries.

Assignment of the bonus points:

Performance bonus: the top 3 teams on the **private** leader board will receive 5 bonus points.

Participation bonus: the 5 teams that submitted the most entries (with different performances) will receive 3 bonus points. Also any team that participated the competition and got non-trivial performance will receive 2 bonus points.

Bonus points are capped at 5.

Please provide the team name on the kaggle competition here __. Leave it blank if you opt not to participate.

```
In [84]: #running this code block will convert this notebook and its outputs into a pdf report.
!jupyter nbconvert --to html /content/gdrive/MyDrive/AI534/IA1-2024.ipynb # you might need to change this path

input_html = '/content/gdrive/MyDrive/Colab Notebooks/IA1-2024.html' #you might need to change this path accord
output_pdf = '/content/gdrive/MyDrive/Colab Notebooks/IA1output.pdf' #you might need to change this path or nam

# Convert HTML to PDF
pdfkit.from_file(input_html, output_pdf)

# Download the generated PDF
files.download(output_pdf)
```

```
[NbConvertApp] Converting notebook /content/gdrive/MyDrive/AI534/IA1-2024.ipynb to html
[NbConvertApp] Writing 920522 bytes to /content/gdrive/MyDrive/AI534/IA1-2024.html
```

```

-----
OSError                                Traceback (most recent call last)
<ipython-input-84-57147974405d> in <cell line: 8>()
      6
      7 # Convert HTML to PDF
----> 8 pdfkit.from_file(input_html, output_pdf)
      9
     10 # Download the generated PDF

/usr/local/lib/python3.10/dist-packages/pdfkit/api.py in from_file(input, output_path, options, toc, cover, css
, configuration, cover_first, verbose)
     46     """
     47
----> 48     r = PDFKit(input, 'file', options=options, toc=toc, cover=cover, css=css,
     49                 configuration=configuration, cover_first=cover_first, verbose=verbose)
     50

/usr/local/lib/python3.10/dist-packages/pdfkit/pdfkit.py in __init__(self, url_or_file, type_, options, toc, co
ver, css, configuration, cover_first, verbose)
     42         css=None, configuration=None, cover_first=False, verbose=False):
     43
----> 44         self.source = Source(url_or_file, type_)
     45         self.configuration = (Configuration() if configuration is None
     46                             else configuration)

/usr/local/lib/python3.10/dist-packages/pdfkit/source.py in __init__(self, url_or_file, type_)
     18
     19         if self.type == 'file':
----> 20             self.checkFiles()
     21
     22         def isUrl(self):

/usr/local/lib/python3.10/dist-packages/pdfkit/source.py in checkFiles(self)
     38         else:
     39             if not hasattr(self.source, 'read') and not os.path.exists(self.source):
----> 40                 raise IOError('No such file: %s' % self.source)
     41
     42         def isString(self):

OSError: No such file: /content/gdrive/MyDrive/Colab Notebooks/IA1-2024.html

```